

✓ Name: G.RISHIKA

ROLL NO: 2303A52197

BATCH: 35

```
def calculate_rectangle_properties(length, width):
    """
    Calculate and print the area and perimeter of a rectangle.

    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.

    Returns:
    None
    """
    area = length * width
    perimeter = 2 * (length + width)

    print("Area of Rectangle:", area)
    print("Perimeter of Rectangle:", perimeter)

# Calling the function with different rectangle dimensions
calculate_rectangle_properties(5, 10)
calculate_rectangle_properties(7, 12)
calculate_rectangle_properties(10, 15)
```

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

```
"""
Task 2: Refactoring - Optimizing Loops and Conditionals
This program compares a legacy nested-loop search with an optimized set lookup.
It prints results and execution times for both approaches.
"""

import time

def legacy_search(names, search_names):
    """
    Perform search using nested loops (inefficient approach).

    Parameters:
    names (list): List of available names.
    search_names (list): Names to search for.

    Returns:
    float: Execution time of the legacy search.
    """
    start = time.time()

    for s in search_names:
        found = False
```

```

        for n in names:
            if s == n:
                found = True
        if found:
            print(f"{s} is in the list")
        else:
            print(f"{s} is not in the list")

    end = time.time()
    return end - start

def optimized_search(names, search_names):
    """
    Perform search using set lookup (optimized approach).

    Parameters:
    names (list): List of available names.
    search_names (list): Names to search for.

    Returns:
    float: Execution time of the optimized search.
    """
    start = time.time()

    names_set = set(names) # Convert list to set for faster lookup

    for s in search_names:
        if s in names_set:
            print(f"{s} is in the list")
        else:
            print(f"{s} is not in the list")

    end = time.time()
    return end - start

# Input data
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

print("---- Legacy Code Output ----")
legacy_time = legacy_search(names, search_names)

print("\n---- Optimized Code Output ----")
optimized_time = optimized_search(names, search_names)

# Performance comparison
print("\n---- Performance Comparison ----")
print(f"Legacy Execution Time: {legacy_time:.8f} seconds")
print(f"Optimized Execution Time: {optimized_time:.8f} seconds")

print("\nComparison Table")
print("-----")
print("Method          | Technique Used | Execution Time")
print("-----")
print(f"Legacy Code      | Nested Loops   | {legacy_time:.8f}")
print(f"Optimized Code   | Set Lookup     | {optimized_time:.8f}")
print("-----")

---- Legacy Code Output ----
Charlie is in the list
Eve is not in the list
Bob is in the list

---- Optimized Code Output ----
Charlie is in the list
Eve is not in the list

```

Bob is in the list

---- Performance Comparison ----

Legacy Execution Time: 0.00002050 seconds

Optimized Execution Time: 0.00001359 seconds

Comparison Table

Method	Technique Used	Execution Time
Legacy Code	Nested Loops	0.00002050
Optimized Code	Set Lookup	0.00001359

"""

Task 3: Refactoring - Extracting Reusable Functions

This program refactors a legacy script by extracting repeated tax and total price calculations into a reusable function.

"""

```
def calculate_total(price):
```

```
    """
```

```
    Calculate the total price including tax.
```

```
    Parameters:
```

```
    price (float): The base price of the item.
```

```
    Returns:
```

```
    float: Total price including tax.
```

```
    """
```

```
    tax = price * 0.18
```

```
    total = price + tax
```

```
    return total
```

```
# Using the reusable function for different prices
```

```
price1 = 250
```

```
total1 = calculate_total(price1)
```

```
print("Total Price:", total1)
```

```
price2 = 500
```

```
total2 = calculate_total(price2)
```

```
print("Total Price:", total2)
```

```
Total Price: 295.0
```

```
Total Price: 590.0
```

"""

Task 4: Refactoring - Replacing Hardcoded Values with Constants

This program replaces magic numbers with named constants to improve readability and maintainability.

"""

```
# Constants
```

```
PI = 3.14159
```

```
RADIUS = 7
```

```
# Calculations using constants
```

```
area = PI * (RADIUS ** 2)
```

```
circumference = 2 * PI * RADIUS
```

```
print("Area of Circle:", area)
```

```
print("Circumference of Circle:", circumference)
```

```
Area of Circle: 153.93791
```

```
Circumference of Circle: 43.98226
```

```
"""
Task 5: Refactoring - Improving Variable Naming and Readability
This program calculates the area of a triangle using clearer
variable names and explanatory comments.
"""

# Base of the triangle
base = 10

# Height of the triangle
height = 20

# Calculate the area of the triangle using the formula:
# Area = (base * height) / 2
area_of_triangle = base * height / 2

# Print the calculated area
print(area_of_triangle)
```

100.0

```
"""
Task 6: Refactoring - Removing Redundant Conditional Logic
This program refactors repeated grading logic into a reusable function.
"""

def calculate_grade(marks):
    """
    Determine and print the grade based on student marks.

    Parameters:
    marks (int or float): The marks obtained by the student.

    Returns:
    None
    """
    if marks >= 90:
        print("Grade A")
    elif marks >= 75:
        print("Grade B")
    else:
        print("Grade C")

# Test the function with different marks
calculate_grade(85)
calculate_grade(72)
```

Grade B
Grade C

```
"""
Task 7: Refactoring - Converting Procedural Code to Functions
This program refactors procedural logic into separate functions
for input, processing, and output.
"""

def get_input():
    """
    Get a number from the user.

    Returns:
    int: The number entered by the user.
    """
    number = int(input("Enter number: "))
    return number
```

```
def calculate_square(number):  
    """  
    Calculate the square of a number.  
  
    Parameters:  
        number (int): The number to be squared.  
  
    Returns:  
        int: Square of the number.  
    """  
    return number * number  
  
def display_result(result):  
    """  
    Display the calculated square.  
  
    Parameters:  
        result (int): The squared value to display.  
  
    Returns:  
        None  
    """  
    print("Square:", result)  
  
# Main program flow  
num = get_input()  
square = calculate_square(num)  
display_result(square)
```

Enter number: 34
Square: 1156

```
"""  
Task 8: Refactoring - Optimizing List Processing  
This program replaces a manual loop with a list comprehension  
to calculate squares of numbers.  
"""  
  
# Original list of numbers  
nums = [1, 2, 3, 4, 5]  
  
# Using list comprehension to generate squares  
squares = [n * n for n in nums]  
  
# Display the result  
print(squares)  
  
[1, 4, 9, 16, 25]
```

